Video Synopsis Lecture 12: Support Vector Machines | ML-005 Lecture 12 | Stanford University | Andrew Ng

Video Contents

Optimization Objective **00:00 – 14:46**

Professor Ng starts with the logistic regression cost function to introduce the SVM cost function and shows that this cost function introduces a margin between the two classes. This comparison also builds the intuition around the C-parameter used in SVM's to apply regularization. He shows that C and λ can be thought of as inversely proportional. The video concludes with the observation that, despite the similarity in cost functions between SVMs and Logistic Regression, SVMs do not output a probability of class membership but a direct prediction of class membership.

Large margin intuition **14:47 – 25:21**

Professor Ng starts with the cost function introduced earlier and explains that, although SVMs will make a decision based on signal $\theta^T x(i)$ being smaller or greater than 0, the cost function used imposes a stricter condition during training, namely that $\theta^T x(i)$ be smaller than -1 for negative and greater than +1 for positive examples. This is made explicit by setting the C-parameter to a high value, in which case a graphical representation shows that the SVM will try to choose the decision boundary such that the largest margin possible is created between the decision boundary and both the nearest positive and negative training samples without misclassifying a single instance. The video concludes by showing that for lower (and more realistic) values of the C-parameter, some instances will be misclassified, but that the resulting margin is likely to be wider and thus more robust when used on out-of-sample data.

SAFE SKIP! Mathematics behind large margin classification **25:22 – 45:03**

Professor Ng starts with a review of vector inner (dot) products as this operator plays an important role in SVMs. The vector inner product is then used to show that for the hard-margin version of the SVM (large C), the constraints imposed on the cost function optimisation problem can be written as a projection of each of the data points onto the weight vector. This is then used to show that a poorly chosen boundary (line in this case) will result in the two conditions ($p^i\|\theta\| \geq 1$ for y=1 and $p^i\|\theta\| \leq -1$ for y=0) requiring θ to be large. But that is the quantity we want to minimise, and hence the SVM will choose a different, more favourable, boundary. It turns out that the best solution is one where the nearest data points lie as far as possible from the decision boundary, which explains the term 'large margin classifier'.

Kernels – I **45:04 – 1:00:48**

Professor Ng starts this video on Kernels by briefly referring to the use of non-linear (higher order) features to make non-linearly separable data sets linearly separable and then introduces a new type of feature that will provide an output based on the similarity of a certain data point ('landmark') with the data point considered. This similarity function is introduced as the Gaussian Kernel and it is shown that its value is close to 1 for points near the landmark and close to 0 for points far away from the landmark. The influence of the Gaussian Kernel parameter σ is introduced as having a narrowing effect on the Gaussian bell shape as σ is reduced. The video concludes by showing that with a few of these landmarks, a complex decision boundary can be created.

Kernels – II **1:00:49 – 1:16:30**

Professor Ng shows that the landmarks are chosen equal to the points in the training data set. I.e., for N data samples, you will end up with N landmarks, which means that the original data vector can be represented with a new feature vector consisting of the chosen Kernel applied to each of the training data samples. This means that the original optimisation problem applied to $\theta^T x(i)$ is now applied to $\theta^T f(i)$. The video concludes by discussing the intuition around the hyper-parameters C and σ. Using a large C, which can be thought of as being inversely proportional to the regularization parameter λ, will generally result in low bias and high variance whereas a smaller C will result in higher bias and lower variance. Hence, C can be used to find the optimum in the bias-variance trade-off. The σ in the Gaussian Kernel widens or narrows the bell-shape and this can be interpreted as smoothing the effect of landmark for higher σ. Such smoothing is associated with lower variance and higher bias. Lower σ will result in the opposite: higher variance and lower bias. Hence, also σ can be used in a bias-variance trade-off / optimisation.

Using an SVM **1:16:31 – 1:37:34**

In the lecture, Professor Ng provides some practical advice regarding the use of SVMs. As he points out, the algorithms used to optimise the SVM cost function are highly complex and you are ill advised to try writing your own algorithm. What is important is the choice of hyper parameters such as C, the kernel, and potential parameters associated with the kernel, such as σ for the Gaussian Kernel. An interesting reference to complexity is made: when you have a data set with few training samples but many features, you are likely best of with a linear kernel as a more complex kernel would likely result in overfitting of the data. For certain kernels, such as the Gaussian kernel it is important to scale the features such that features with high ranges do not dominate the features with smaller ranges.

Professor Ng mentions that not all functions are suitable as a kernel and mentions Mercer's theorem without further explanation (which is fine for our purposes). Other kernels mentioned, are the Polynomial kernel which raises the inner product of the data points with a landmark to the n-th power (potentially after adding a constant to the result of the inner product) and some more esoteric kernels that will hardly ever be used in practice. The ability of SVMs to perform multi-class classification is raised briefly. Most implementations will do this out of the box, but the one vs all approach can also be used. This entails training an SVM for each of the classes in your data set, where each SVM classifies the respective class versus all other classes in the set. Classification is performed by calculating the $\theta^T x(i)$ for each of these SVMs and deciding on the class corresponding with the SVM that yielded the highest value of $\theta^T x(i)$.

Finally, some advice is given on the use of SVMs and Logistic Regression for multiple scenarios:

- When you have many features (say 10,000) and not so many data points (10 .. 1000), you are likely best of with a linear kernel or with logistic regression. This is for the aforementioned reason that you do not have enough data to fit a complex model and is in line with what we have learned about the theory of generalization.
- When you fewer features (1 up to 1000) and a reasonable amount of training data (10 – 10,000), SVMs are in their element and you can use a Gaussian kernel to learn potentially non-linear relationships. Note that if your number of features is toward the higher end of the above range, the number of training samples should also be toward the higher end of the range.
- When you have few features, but a large training set (say 50,000+), the SVM algorithm may start struggling as it uses all data points in the optimisation. In this case you are advised to use logistic regression or an SVM with a linear kernel to reduce the computational burden of optimisation and manually add some new features to improve linear separability.